

GRA41522

Final project

Bannerid:

1084518

Anonymized GitHub repository:

<https://github.com/ANNMS0/GRA41522Prjct>

Course:

GRA4152 Object Oriented Programming with Python

Contents

Preamble	1
1. Implementing a VAE	1
1.1 BiCoder	1
1.2 Encoder.....	2
1.3 Decoder.....	3
1.4 VAE.....	3
2 DataLoader Class	4
2.1 DataLoader	4
3 OOP Core Concepts	6
3.1 Why OOP?.....	6
3.2 Inheritance	6
3.3 Abstract Methods.....	7
3.4 Polymorphism.....	7
3.5 Overriding.....	7
3.6 Decorators.....	8
4 Training & Visualisation	8
4.1 Training Script.....	8
4.2 Visualisations	8
5 Appendix	9

PREAMBLE

The implementation in this project follows the KISS principle: keep it simple student, reuse what is already given and match the expected behaviour of the assignment as closely as possible. The code is split into small modules and a dedicated data folder used to separate source files from downloaded datasets and keep the project structure clear. Where possible, the Professor's files were reused directly, with only minimal import fixes and filename changes so that the generated plots and outputs follow the same conventions as in the examples. Docstrings are added to clarify the public interfaces of the main classes and functions. For robustness, the data loader accepts multiple names for the black-and-white dataset and for the mnist_color data a single preprocessing variant (m4) was chosen to ensure consistent behaviour across runs.

The report follows the numbering and questions of the assignment, all remaining design choices are discussed in their respective sections, the README file or the code itself. It is to my understanding that the report is not supposed to explain VAE or the mathematical theory behind it, but rather to display knowledge of Object-Oriented Programming (OOP) and the ability to apply these concepts properly in solving this assigned problem per the instructions.

1. IMPLEMENTING A VAE

1.1 BiCODER

This class implements the common interface for both the encoder and the decoder. BiCoder inherits from `tf.keras.layers.Layer` and `ABC`, ensuring that all subclasses behave as standard TensorFlow layers while also enforcing an abstract architecture. Two abstract methods are defined within this class:

- `build_network()`, which mandates each subclass to override to construct the correct neural network for its role and dataset and
- `call()`, which defines the forward pass.

This follows the assignment requirement to use inheritance, abstract classes, overriding and polymorphism. The encoder and decoder share the same structural contract, but differ in implementation, allowing the VAE to interact with them uniformly.

The constructor stores the dataset type (`bw` or `color`), so subclasses adapt their internal architecture as demanded (MLP for black and white, CNN for colour). This design avoids duplicated logic in child classes.

Justification of design choice

I used a shared parent class to keep the implementation simple and follow the KISS principle mentioned in the preamble. The Professor's architecture already treats encoder and decoder symmetrically, so defining a common abstract base class avoids redundancy and matches the expected modelling strategy. Inheriting from `tf.keras.layers.Layer` is essential as it ensures that all subclasses integrate correctly with TensorFlow's computation graph, variable tracking and training loop, so that the VAE can be trained and used in a typical Keras workflow. Abstract methods force subclasses to implement only the parts that differ (their networks in this case), which improves clarity and makes debugging easier and consistent with the instructions in the project description. Docstrings were used to make the public interface easy to understand, as advised during the lectures.

1.2 ENCODER

The `Encoder` class specializes the abstract `BiCoder` by implementing the probabilistic encoder $q_{\phi}(z | x)$. It inherits from `BiCoder` and overrides both `build_network()` and `call()`. The constructor forwards the dataset flag (`bw` or `color`) to the parent class and immediately calls `build_network()`, so each encoder instance is created with a valid underlying network. The `build_network()` method selects between the provided architectures (`encoder_mlp` for black and white data and `encoder_conv` for color images), assigning the choice to `self.network`. The `call()` method applies this network to an input mini-batch `x` and splits the output into two equal halves, returning `z_mean` and `z_logvar`, which parameterize a diagonal Gaussian approximate posterior.

Justification of design choice

I used the encoder as a thin wrapper around the Professor's `encoder_mlp` and `encoder_conv` functions from `neural_networks.py` to keep things simple and stay close to the assignment's reference implementation. Using `build_network()` to select the architecture based on the dataset flag demonstrates overriding and polymorphism, while the shared interface inherited from `BiCoder` allows the VAE to treat any encoder instance uniformly. Returning only the latent parameters and leaving sampling to the VAE class separates responsibilities clearly: the encoder

computes $(\mu, \log \sigma^2)$ and the VAE handles the reparameterization and loss computation. The explicit `ValueError` for unsupported dataset types is a defensive design choice that makes incorrect usage fail fast and easier to debug, it is particularly useful when generating samples from the prior, where no dataset is loaded to catch potential configuration mistakes.

1.3 DECODER

The `Decoder` class is the counterpart to the `Encoder` and specializes the `BiCoder` interface by implementing the generative mapping $p\theta(x | z)$, completing the second half of the VAE. It inherits from `BiCoder` and reuses the same structure: the constructor forwards the dataset flag and calls `build_network()`, while the actual network comes from the Professor's `decoder_mlp (bw)` or `decoder_conv (color)`. The `call()` method applies this network to a latent vector z and returns a reconstruction in the original data space.

Justification of design choice

The `Decoder` mirrors the `Encoder` on purpose, keeping both sides structurally symmetric makes the architecture easier to follow and avoids unnecessary divergence from the assignment's reference design. Since the task does not require custom architectures, wrapping the provided decoder functions keeps the implementation simple, predictable and fully consistent with the expected output shapes. The `Decoder` performs a deterministic forward pass only, leaving sampling and loss construction to the VAE, which keeps responsibilities cleanly separated. This symmetry, combined with overriding only the components that differ (the underlying network), highlights the intended use of inheritance and polymorphism in the project.

1.4 VAE

The `VAE` class subclasses `tf.keras.Model` and ties the whole system together. It owns an `Encoder` and a `Decoder` instance (composition) and is responsible for performing the reparameterization trick and computing the ELBO. The constructor receives the dataset flag (`bw` or `color`), creates the corresponding encoder and decoder, and sets the latent dimensionality according to the project specification (20 for BW, 50 for color). It also defines the decoder noise level `sigma_x` and its logarithm, which are later used when evaluating the Gaussian log-likelihood.

The `call()` method implements the full forward pass used during training. First, it encodes an input mini-batch into x into (z_mean, z_logvar) , then applies `reparameterize()` to

obtain a latent sample z . This sample is decoded to x_{μ} , the mean of the reconstruction distribution. Using the professor's `log_diag_mvn` and `kl_divergence` functions, the method computes the reconstruction log-probability and the KL divergence term, averages them over the batch and stores them in `self.recon_logprob`, `self.kl_loss` and `self.elbo`. The final quantity returned by `call()` is `self.vae_loss`, defines as negative ELBO.

The `train()` method, decorated with `@tf.function`, wraps a single training step in a `tf.GradientTape`, calls the model on the batch, computes gradients of `vae_loss` with respect to all trainable variables and applies an optimizer update. Convenience methods `encode()`, `decode()` and `sample_prior()` expose the encoder-decoder behaviour without re-running the full loss computation, which is useful for latent-space visualization and generating samples from the prior.

Justification of design choice

I placed the reparameterization, loss construction and training logic inside the VAE class to keep the encoder and decoder focused on their single responsibilities (producing latent parameters and reconstructions). Using composition (`self.encoder`, `self.decoder`) together with inheritance from `tf.keras.Model` allows the model to integrate seamlessly with the Keras training ecosystem while still demonstrating the OOP concepts required in the assignment. The `@tf.function` decorator on `train()` follows TensorFlow best practice by compiling the training step for efficiency and the explicit storage of `elbo`, `kl_loss` and `recon_logprob` makes it straightforward to log and inspect each component during experiments. The design stays close to the structure suggested in the project description, in line with the KISS principle used throughout the implementation.

2 DATALOADER CLASS

2.1 DATALOADER

The `DataLoader` class encapsulates all dataset handling for the project: downloading, preprocessing and packaging the MNIST data into a `tf.data.Dataset` for training, plus separate arrays for test-time visualizations. The constructor takes a dataset name (`bw/mnist_bw` or `color/mnist_color`) and a batch size, prepares a dedicated `data/` directory and then calls a sequence of private helper methods: `_set_paths_and_urls()`, `_ensure_data_dir()`, `_download_if_needed()`, `_load_data()` and

`_build_tf_dataset()`. This design keeps the public interface small as external code only needs to initialize a `DataLoader` object and call `get_training_data()`, `get_test_data()` or `get_test_labels()`.

The test data and labels are exposed, because they are required for the t-SNE latent space visualization and reconstruction plots performed after training, as specified in the assignment.

For the black and white data, `_set_paths_and_urls()` points to the `.npz` files and `_load_data()` simply uses `np.load`. For the colour data, it selects the `.pkl` files and loads them with `pickle`, then fixes the version to `m4` as a single deterministic choice among the five available variants. The `_download_if_needed()` method implements the project's requirement to use `wget` from Python: it loops over the URLs and calls `subprocess.run(["wget", url, "-O", path], check=True)` only when the local file is missing. The `_build_tf_dataset()` method performs all preprocessing steps: it converts the arrays to `float32`, scales values to the interval `[0, 1]` if necessary, vectorizes the bw images to 784-dimensional vectors when `self.dset` is "bw" and finally builds a shuffled, batched `tf.data.Dataset` using `tf.data.Dataset.from_tensor_slices()`.

Justification of design choice

I started the project from the `DataLoader` as placing all dataset logic inside it keeps the training script and VAE model independent of file formats, download details and preprocessing decisions, which follows encapsulation and separation-of-concerns principles emphasized in the course. Supporting multiple names for each dataset ("`bw`"/"`mnist_bw`", "`color`"/"`mnist_color`") makes the interface more robust and faster to use without adding complexity. Fixing the color version to `m4` ensures reproducible behaviour while still respecting the assignment's freedom to pick any of the available variants. I tried each version and decided to stick with `m4` knowing it wasn't the same as the Professor's who used `m1` going by my testing. The use of small helper methods for paths, downloads, loading and preprocessing keeps each step easy to test and debug, and reusing `wget`, URLs and preprocessing choices from the professor's material is consistent with the KISS principle that I tried to apply throughout the project.

3 OOP CORE CONCEPTS

3.1 WHY OOP?

This project fits an object-oriented design almost by necessity: we have several moving parts (encoder, decoder, VAE wrapper and `DataLoader_` that need clean interfaces and minimal duplication. The assignment and lectures emphasized keeping components focused on their own responsibilities and reusing structure where possible, which aligns directly with the core OOP ideas. Using inheritance, abstract methods, polymorphism, overriding and decorators let me match the architecture suggested by the Professor while keeping the code organised, testable and easy to extend.

3.2 INHERITANCE

I utilized inheritance to express shared behaviour and structure between the encoder, decoder and parent VAE components without having to repeat their logic. `BiCoder` inherits from `tf.keras.layers.Layer`, which lets TensorFlow treat all subclasses as proper Keras layers (variable tracking, saving/loading, integration with the training loop). The `Encoder` and `Decoder` then inherit from `BiCoder`, reusing the common interface while overriding only what differs, such as `build_network()` and `call()`.

Examples from this project:

- `BiCoder -> tf.keras.layers.Layer`: integrates the abstract interface with TensorFlow's layer system.
- `Encoder/Decoder -> BiCoder`: both reuse the shared constructor logic and abstract methods while implementing different architectures (MLP vs CNN).
- `VAE -> tf.keras.Model`: allows the entire model to plug straight into the Keras ecosystem for training, gradient computation and exporting.

This structure keeps the codebase consistent with the design suggested in the assignment: shared behaviour lives in one place, and components only override the parts that genuinely differ. It reduces duplication and makes the architecture easier to maintain, test and extend.

3.3 ABSTRACT METHODS

I used abstract methods to enforce the structure of the encoder and decoder without fixing their internal architectures. The `BiCoder` class declares `build_network()` and `call()` as abstract, which forces every subclass to implement them. This removes ambiguity about what each component must provide and keeps the interface stable for the VAE.

Examples from this project:

- `class BiCoder(..., ABC)` defines abstract `build_network()` and `call()`.
- `Encoder` and `Decoder` implement these methods with dataset-specific architectures.

This ensures that all variants are compatible with the VAE and that only the parts that differ (MLP vs CNN, shapes, outputs) must be implemented, which follows the assignment's push for clean interfaces and minimal duplication.

3.4 POLYMORPHISM

Polymorphism appears naturally through the shared `BiCoder` interface. Both `Encoder` and `Decoder` respond to the same methods (`build_network()`, `call()`, `__init__`) but behave differently depending on the dataset (bw vs color) and architecture (MLP vs CNN). The VAE does not need to know which encoder or decoder it is using, only that it exposes the expected interface.

Examples from this project:

- `self.encoder(x)` and `self.decoder(z)` in the VAE: the VAE treats both components uniformly.
- `build_network()` selects different architectures internally based on `self.dset`.
- `decode()` and `encode()` methods on the VAE delegate to whichever subclass is present.

This supports the “replaceable components” idea in the assignment and lets me keep the VAE logic independent of architectural choices.

3.5 OVERRIDING

Overriding is used to customize behaviour inherited from base classes. `Encoder` and `Decoder` override `build_network()` and `call()` to plug in the correct MLP/CNN networks while still matching the interface defined by `BiCoder`. The VAE overrides `call()` from `tf.keras.Model` to compute ELBO, KL divergence and reconstruction log-probabilities instead of producing a simple forward pass.

Examples from this project:

- Encoder/Decoder override `build_network()` and `call()`.
- VAE overrides `call()` and adds custom behaviour such as setting `self.elbo`, `self.kl_loss`, etc.

This keeps the shared interface in one place while letting each component define its actual behaviour, consistent with the assignment requirement to demonstrate overriding.

3.6 DECORATORS

I used the `@tf.function` decorator on the VAE's `train()` method to follow TensorFlow's recommended training-loop style. The decorator compiles the function into a TensorFlow graph, which significantly improves performance and keeps the training step consistent across runs.

Examples from this project:

- `@tf.function` on `VAE.train()`

The decorator demonstrates the project's requirement to show knowledge of decorators while also matching the expected TensorFlow/Keras workflow.

4 TRAINING & VISUALISATION

4.1 TRAINING SCRIPT

The `train_vae.py` file acts as the entry point for running the system. It constructs the `DataLoader` and VAE objects. Iterates through the batches and saves the plots required by the assignment. The script is intentionally short and simple, because the OOP structure places all real responsibilities inside the classes. This keeps the workflow close to the layout shown in the project description and lets the script focus solely on orchestration.

4.2 VISUALISATIONS

I have added visualisations to the appendix to help showcase the structure of this program, the visualisations required by point f) testing your code can be created by simply running it and as such I don't think they are worthy putting in this report that is supposed to be compact and minimalistic.

5 APPENDIX

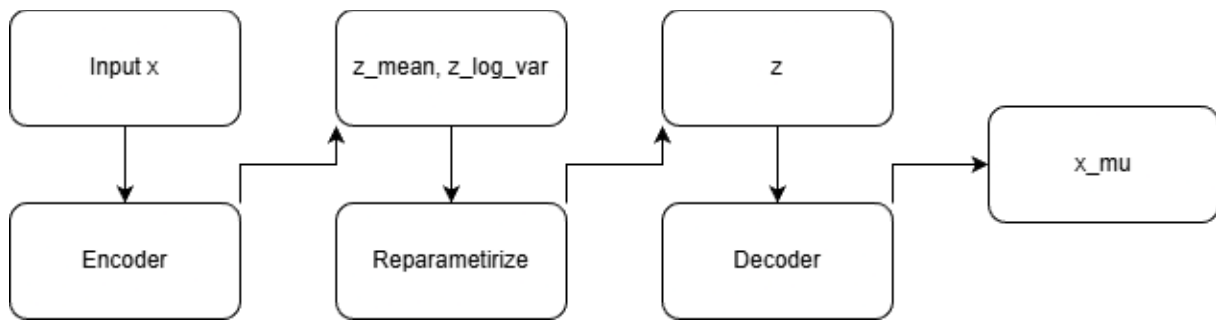


Fig 1. Architecture diagram (simplified)

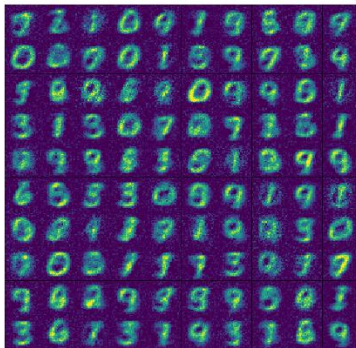


Fig 2. Nonstandard 1 epoch bw posterior sample result, surprisingly decent for 1 epoch

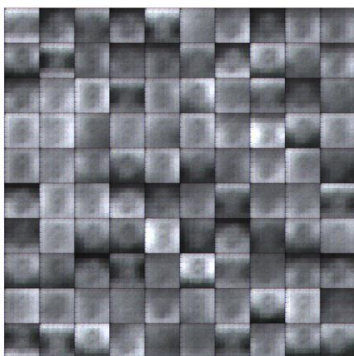


Fig 3. 1 epoch color posterior sample result, looks closer to a black and white zoom in on a straw basket